

EPIC 09- Task 01: Requirements & Architecture

Real-World System: Rapido / Bike Taxi System

The system allows users to book bike rides, drivers to accept rides, track locations, make payments, and receive notifications.

1. Users

Users are customers who use the application to:

- Register and log in.
- Enter pickup and destination locations.
- Request a bike ride.
- View available drivers.
- Track the driver.
- Cancel for a ride.
- Make payments.
- View ride history.
- Receive notifications.

2. Drivers

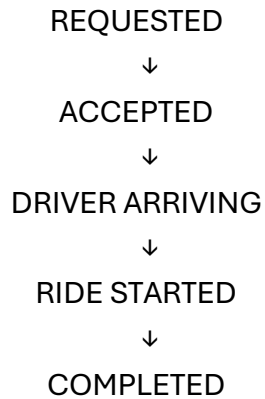
Drivers provide bike-taxi services.

They can:

- Register and manage their profile.
- Go online/offline.
- Share their current location.
- Receive ride requests.
- Accept or reject rides.
- Navigate to the pickup location.
- Start and complete rides.
- View earnings.
- Receive payment information.

3. Rides

The Ride system manages the complete ride lifecycle.



It should also support:

CANCELLED

The system stores:

- User ID
- Driver ID
- Pickup location
- Destination
- Ride status
- Fare
- Ride start/end time

4. Payments

The Payment system handles:

- Fare calculations.
- Payment processing.
- Payment confirmation.
- Refunds.
- Payment history.

Supported payment methods can include:

- UPI
- Credit/Debit Card
- Wallet
- Cash

5. Locations

The Location system manages:

- User pickup location.
- Driver's current GPS location.
- Destination.
- Driver location updates.
- Distance calculation.
- Estimated time of arrival.

6. Notifications

The Notification system sends:

- Ride request notifications.
- Driver acceptance notifications.
- Driver-arrival notifications.
- Ride-start notifications.
- Ride completion notifications.
- Payment confirmation.
- Cancellation notifications.

Functional Requirements

The system should provide the following functionality:

1. Users can register and log in.
2. Users can request a bike ride.
3. The system can identify nearby available drivers.
4. Drivers can accept or reject ride requests.

5. Users can track their drivers.
6. Drivers can update their location.
7. Users can cancel rides.
8. The system calculates the ride fare.
9. Users can make payments.
10. The system records payment transactions.
11. Users and drivers can receive notifications.
12. Users can view their ride history.
13. Drivers can view their completed rides and earnings.
14. The system maintains the status of every ride.
15. Failed payments and cancelled rides must be handled correctly.

Non-Functional Requirements

Performance

- API responses should normally be fast.
- Location updates should be handled efficiently.
- The system should support many users and drivers simultaneously.

Scalability

- Individual microservices should be scalable independently.
- The system should support increasing numbers of rides and users.

Availability

- The system should remain available even if a non-critical service fails.
- Services should support health checks and monitoring.

Security

- User authentication and authorization must be implemented.
- Passwords must be securely stored.
- Payment information must be protected.
- Sensitive information must not be exposed in logs.

Reliability

- Ride requests should not be lost.
- Payment transactions should be handled reliably.
- Service failures should have proper error handling.

Maintainability

- Services should have clearly defined responsibilities.
- APIs should follow consistent REST conventions.
- Logging and monitoring should be implemented.

Data Consistency

- Ride status must remain consistent between services.
- Payment status must be recorded accurately.
- Duplicate payment or ride processing should be prevented.

Final Requirement Summary

